Learning Objectives

Learners will be able to...

- **Define the concept of an iterator and understand its role in traversing collections in C++.**
- **Explain the Iterator in C++ and its key methods: `begin()`, `end()`, `++`, and `*`.**
- **Implement custom iterators to enable iteration over a custom List ADT, such as a vector, list, or doubly linked list.**
- **Understand the differences between basic iterators and more advanced types like bidirectional iterators.**
- **Apply iterators to perform common operations on containers, such as traversal, filtering, and modification while iterating.**
- **Analyze the complexities of iterator operations in C++ and understand their impact on the performance of container manipulations.**

Introduction to Iterators

Iterators

In C++, an iterator is a powerful tool used to traverse containers, such as vectors, lists, and maps. It provides a standardized way to access elements sequentially without exposing the underlying representation of the container.

Understanding the Iterator in C++

C++ iterators are part of the Standard Template Library (STL) and are used similarly to pointers. The key operations they support are:

- `bool operator!=(const iterator&)`: Returns true if there are more elements in the iteration.
- `iterator& operator++()`: Advances the iterator to the next element.
- `T& operator*()`: Returns a reference to the current element in the iteration.
- `void erase(iterator position)`: Removes the element at the specified position (supported in some containers).

Using Iterators with the List ADT

Iterators are particularly useful for iterating over containers like vectors, lists, and doubly linked lists. They provide a clean and concise way to loop through elements, perform operations, and even modify the container during iteration.

Example: Iterating over a List

Create the variable `it` and set its value to the `names.begin()` which is an iterator for the beginning of the list. Iterate as long as `it` is not equal to the last iterator in the last. With each iteration of the loop, increase the value of `it` by 1.

```
int main() {
    std::list<std::string> names = {"Alice", "Bob", "Charlie"};

    for (auto it = names.begin(); it != names.end(); it++) {
        std::cout << *it << std::endl;
    }

    return 0;
}
```

challenge

Try this variation

Using an iterator and the `erase` method, remove all odd numbers from `numbers`, a list that contains all sequential numbers from 0 to 9.

▼ **Solution**

Here is one possible solution:

```
int main() {
    std::list<int> numbers = {0, 1, 2, 3, 4, 5, 6, 7,
        8, 9};

    for (auto it = numbers.begin(); it !=
        numbers.end(); it++) {
        if (*it % 2 != 0) {
            it = numbers.erase(it);
        }
    }

    for (const auto& number : numbers) {
        std::cout << number << " ";
    }

    return 0;
}
```

Create the list `numbers`. Then create an iterator object for the list. Iterate over the list, checking if each element is odd. If so, use `erase` to remove the element from the list.

On this page, we created a `std::list` of names, and then used an iterator to traverse the list and print each name. We also saw how the `erase` method can be used to remove elements from a list.

In the next section, we will explore more advanced iterator types like bidirectional and their capabilities compared to basic iterators.

Bidirectional Iterator

Adding Functionality to an Iterator

The bidirectional iterator concept in C++ extends the basic iterator by providing additional functionality for bidirectional iteration over a collection. It allows you to modify elements in the collection and even insert or erase elements during iteration.

Operations for Bidirectional Iterator

- `bool operator!=`: Returns true if there is a next element in the collection.
- `Iterator& operator++`: Advances the iterator to the next element.
- `bool operator==`: Returns true if there is a previous element in the collection.
- `Iterator& operator--`: Moves the iterator to the previous element.
- `Iterator operator+(int)`: Returns an iterator that is advanced by the specified number of elements.
- `Iterator operator-(int)`: Returns an iterator that is moved backward by the specified number of elements.
- `void operator*=(T&)`: Replaces the element at the iterator's current position with the specified value (using a reference).

Using Bidirectional Iterators

Start off just as before by creating a `std::list` of names.

```
int main() {
    std::list<std::string> names = {"Alice", "Bob", "Charlie"};
}
```

We can iterate over the list like we did on the previous page. Once the forward iteration is done, create another loop and iterate over the list. This time we are going to use `rbegin` and `rend` to create reverse iterators. We are still going to increment the reverse iterator.

```
    std::cout << "Forward iteration:" << std::endl;
    for (auto it = names.begin(); it != names.end(); it++) {
        std::cout << *it << std::endl;
    }

    std::cout << "\nBackward iteration:" << std::endl;
    for (auto it = names.rbegin(); it != names.rend(); it++) {
        std::cout << *it << std::endl;
    }

    return 0;
```

challenge

Try this variation

- Use the `std::advance` method to move bidirectionally through a list.

```
int main() {

    // Create a list of strings
    std::list<std::string> names = {"Alice", "Bob",
        "Charlie"};
    auto listIterator = names.begin();

    // move forward two elements - Charlie
    std::advance(listIterator, 2);
    std::cout << *listIterator << std::endl;

    // move backward two elements - Alice
    std::advance(listIterator, -2);
    std::cout << *listIterator << std::endl;

    return 0;

}
```

▼ What is happening?

> The `std::advance` method allows you to alter an iterator by adding (move forward) or subtracting (moving backward). If you use `2`, then you are moving forward two elements at a time. Using `-2` means you are moving backwards two elements at a time.

We can use the dereference operator (`*`) to modify elements in a collection. Create a list of names where some of the names are not capitalized. With a bidirectional iterator, traverse the list. Check the first letter of each name. If

the letter is lowercase, create a string where the first letter is capitalized. Then, using the `*` operator, replace the element in the list with the capitalized name.

```
int main() {
    std::list<std::string> names = {"alice", "Bob", "charlie"};
    for (auto it = names.begin(); it != names.end(); ++it) {
        char firstLetter = (*it)[0];
        if (std::islower(firstLetter)) {
            (*it)[0] = std::toupper(firstLetter);
        }
    }
    std::cout << "{ ";
    for (const auto& name : names) {
        std::cout << name << " ";
    }
    std::cout << "}" << std::endl;

    return 0;
}
```

Finally, we are going to use the `insert` method to add an element to the list. Create a list of the integers `1`, `3`, and `5`. We are going to insert `2` and `4`. Create an iterator but do not create a loop. Advance the iterator so it points to `1`. Then insert the number `2`. Important, the insert method will add an element and return an iterator pointing to the newly inserted element. So to place the `4` in the proper position, you need to increment the iterator and then insert the missing integer.

```
int main() {
    std::list<int> numbers = {1, 3, 5};

    // Create an iterator
    auto it = numbers.begin();

    // Insert 2 between 1 and 3
    std::advance(it, 1);
    numbers.insert(it, 2);

    // Insert 4 between 3 and 5
    std::advance(it, 1);
    numbers.insert(it, 4);

    std::cout << "{ ";
    for (const auto& num : numbers) {
        std::cout << num << " ";
    }
    std::cout << "}" << std::endl;

    return 0;
}
```

In this example, we use a bidirectional iterator to iterate over a `std::list` in both forward and backward directions, access the indices of the elements, update elements, and add elements to the list. The C++ standard library provides a robust set of iterator operations that allow us to accomplish more tasks than a basic iterator.

Custom Iterator

Creating a Custom Iterator

We have seen how implementing the `List` interface gives us access to an iterator. In this example, we will create a simplified version of a singly linked list that includes an iterator. This iterator will allow us to traverse the list using standard C++ iteration techniques.

▼ **Why create an iterator?**

> In C++, implementing iterators can make it easier to traverse data structures without exposing their internal details. Iterators provide a uniform interface, allowing you to use standard library algorithms and range-based for loops for traversal.

Example: Implementing a Custom Iterator for a Singly Linked List

Start by creating the `Node` class that contains strings. This class remains unchanged when implementing an iterator.

```
class Node {
public:
    std::string data;
    Node* next;

    Node(const std::string& data) : data(data), next(nullptr) {}
};
```

Next, create the `LinkedList` class that will implement an iterator for traversing the list. The `head` attribute and the `add` method will allow us to add nodes to the list. We'll keep the class simple and focus on the iteration.

```
class LinkedList {
    Node* head;

public:
    LinkedList() : head(nullptr) {}

    void add(const std::string& data) {
        Node* newNode = new Node(data);

        if (head == nullptr) {
            head = newNode;
        } else {
            Node* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;
            }
            temp->next = newNode;
        }
    }

};
```

Now, we create the custom iterator class `Iterator` inside the `LinkedList` class. The iterator will hold a pointer to the current node. We can override the necessary operators for iterating over the list (`!=`, `++`, and `*`).

```
    class Iterator {
        Node* currentNode;

    public:
        Iterator(Node* start) : currentNode(start) {}

        // Overloading != operator to compare two iterators
        bool operator!=(const Iterator& other) const {
            return currentNode != other.currentNode;
        }

        // Overloading ++ operator to advance the iterator
        Iterator& operator++() {
            if (currentNode != nullptr) {
                currentNode = currentNode->next;
            }
            return *this;
        }

        // Overloading * operator to dereference the iterator
        std::string& operator*() {
            return currentNode->data;
        }
    };
```

Finally, implement the `begin()` and `end()` methods in the `LinkedList` class. These methods return iterators pointing to the start (`head`) and the end (`nullptr`) of the list. Technically, you must define at least one of these

methods. However, doing so limits what you can do with your custom iterator object. It is a good idea to define both the `begin` and `end` methods.

```
    // Returns an iterator pointing to the first element of the
        list
    Iterator begin() {
        return Iterator(head);
    }

    // Returns an iterator pointing to the past-the-end element
        (nullptr)
    Iterator end() {
        return Iterator(nullptr);
    }
```

Testing the Code

By defining a custom iterator, we can now use standard iteration constructs like for loops to traverse our linked list. In the main function, create a LinkedList object, add names to the list, and use the iterator to print each name.

```
int main() {
    LinkedList names;
    names.add("Alice");
    names.add("Bob");
    names.add("Charlie");

    for (auto it = names.begin(); it != names.end(); ++it) {
        std::cout << *it << std::endl;
    }

    return 0;
}
```

▼ **Code**

> Your code should look like this:
>
> ```
> class Node {
> public:
>     std::string data;
>     Node* next;
>
>     Node(const std::string& data) : data(data),
>         next(nullptr) {}
> };
>
> class LinkedList {
>     Node* head;
>
> public:
> ```

```
    LinkedList() : head(nullptr) {}

    void add(const std::string& data) {
        Node* newNode = new Node(data);

        if (head == nullptr) {
            head = newNode;
        } else {
            Node* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;
            }
            temp->next = newNode;
        }
    }

    class Iterator {
        Node* currentNode;

    public:
        Iterator(Node* start) : currentNode(start) {}

        // Overloading != operator to compare two iterators
        bool operator!=(const Iterator& other) const {
            return currentNode != other.currentNode;
        }

        // Overloading ++ operator to advance the iterator
        Iterator& operator++() {
            if (currentNode != nullptr) {
                currentNode = currentNode->next;
            }
            return *this;
        }

        // Overloading * operator to dereference the
        iterator
        std::string& operator*() {
            return currentNode->data;
        }
    };

    // Returns an iterator pointing to the first element of
        the list
    Iterator begin() {
        return Iterator(head);
    }

    // Returns an iterator pointing to the past-the-end
        element (nullptr)
    Iterator end() {
        return Iterator(nullptr);
    }
};

int main() {
    LinkedList names;
    names.add("Alice");
```

```
    names.add("Bob");
    names.add("Charlie");

    for (auto it = names.begin(); it != names.end(); ++it)
        {
        std::cout << *it << std::endl;
    }

    return 0;
}
```

challenge

Try this variation

Change the main method so that you traverse the list using a range-based for loop. This is possible because we have defined `begin()` and `end()` in the `LinkedList` class.

```
int main() {
    LinkedList names;
    names.add("Alice");
    names.add("Bob");
    names.add("Charlie");

    for (const auto& name : names) {
        std::cout << name << std::endl;
    }

    return 0;
}
```

▼ **What is happening?**

> Another benefit to creating custom iterators in C++ is that you can use range-based for loops, which are more concise and easier to read than traditional loops.

In this example, we used a custom iterator to traverse the linked list using both a traditional for loop and a range-based loop. This shows how iterators provide flexibility and make your data structures compatible with standard C++ iteration patterns.

Next, we will explore more advanced iterator concepts, including handling concurrent modifications during iteration and using iterators in multi-threaded environments.

Iterators and Concurrency

Handling Concurrent Modifications and Multi-Threading with Iterators

When iterating over a collection in C++, it's important to consider how concurrent modifications to the collection can affect the iteration process. Additionally, when working in multi-threaded environments, where multiple threads might access and modify a collection concurrently, special care must be taken to ensure thread safety.

Concurrent Modifications

A concurrent modification occurs when a collection is modified while it is being iterated over. This can lead to unpredictable behavior and, in most cases, will result in undefined behavior. To avoid this, you should not modify the collection directly while iterating, except through the iterator's own methods if the container supports such safe modifications.

Multi-Threading with Iterators

In a multi-threaded environment, multiple threads can access and modify a shared collection concurrently. This can lead to issues such as data corruption and inconsistent state if not handled properly. To ensure thread safety:

- Use thread-safe collections provided by libraries such as boost or the concurrent data structures in the C++ standard library.
- Synchronize access to the collection using mutexes or locks.
- Consider using iterators designed to be used in a concurrent context, if available.

Let's start by looking at how iterators can be problematic in a multi-threaded environment. The code below creates a vector of strings and adds the first element. The first thread uses a loop to add three more elements to the vector. The second thread uses an iterator to iterate over the vector printing each element.

```
int main() {
    std::vector<std::string> list;
    list.push_back("item1");

    std::thread t1([&list]() {
        for (int i = 0; i < 3; i++) {
            list.push_back("item" + std::to_string(i + 2));

        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }
    });

    std::thread t2([&list]() {

        std::this_thread::sleep_for(std::chrono::milliseconds(50));

        for (const auto& item : list) {
            std::cout << item << std::endl;

        std::this_thread::sleep_for(std::chrono::milliseconds(50));
        }
    });

    t1.join();
    t2.join();

    return 0;
}
```

▼ **Code quality**

> The code used in these examples explicitly tries to show the issue of concurrent modification. This is bad practice in production code and is purposely written this way to make a point. You should not view these examples as idiomatic C++.

This code can produce undefined behavior. The first thread is adding elements to the vector, while the second thread is looping over the vector and printing its elements. However, you may see only `item1` printed. There are ways to avoid this. One way is to replace the vector with a thread-safe collection or to ensure proper synchronization. Replace the `main` method with the code below.

```
int main() {
    std::vector<std::string> list;
    list.push_back("item1");
    std::mutex mtx;
    std::atomic<bool> done{false};  // Flag to signal when t1 is
        done

    // Thread 1: Add items to the vector
    std::thread t1([&list, &mtx, &done]() {
        for (int i = 0; i < 3; i++) {
            {
                std::lock_guard<std::mutex> lock(mtx);
                list.push_back("item" + std::to_string(i + 2));
            }

        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }
        done = true;  // Signal that t1 is finished
    });

    // Thread 2: Print items from the vector
    std::thread t2([&list, &mtx, &done]() {
        while (!done) {  // Check if t1 is still running
            std::vector<std::string> current_items;
            {
                std::lock_guard<std::mutex> lock(mtx);
                current_items = list;  // Make a copy while
        holding the lock
            }

            // Process the copied items without holding the lock
            for (const auto& item : current_items) {
                std::cout << item << std::endl;
            }


        std::this_thread::sleep_for(std::chrono::milliseconds(50));
        }

        // One final check after t1 is done to ensure we get all
        items
        std::lock_guard<std::mutex> lock(mtx);
        for (const auto& item : list) {
            std::cout << item << std::endl;
        }
    });

    t1.join();
    t2.join();

    return 0;
}
```

In this version, we use a `mutex` which is a synchronization mechanism to ensure that only one thread is working on a shared resource at a time. Essentially the `mutex` is locking and unlocking the vector in each thread.

This is not only thread-safe but also allows us to see all the elements in the vector as they are added.

Key things to remember:

- Iterators provide a standardized way to traverse collections in C++. Use iterators to iterate over collections rather than traditional for loops, as they provide a cleaner and more flexible way to traverse elements.
- Proper handling of concurrent modifications and thread safety is crucial when iterating over collections in multi-threaded environments. Use synchronization mechanisms like mutexes or thread-safe collections to avoid undefined behavior.

Formative Assessment 1

Formative Assessment 2